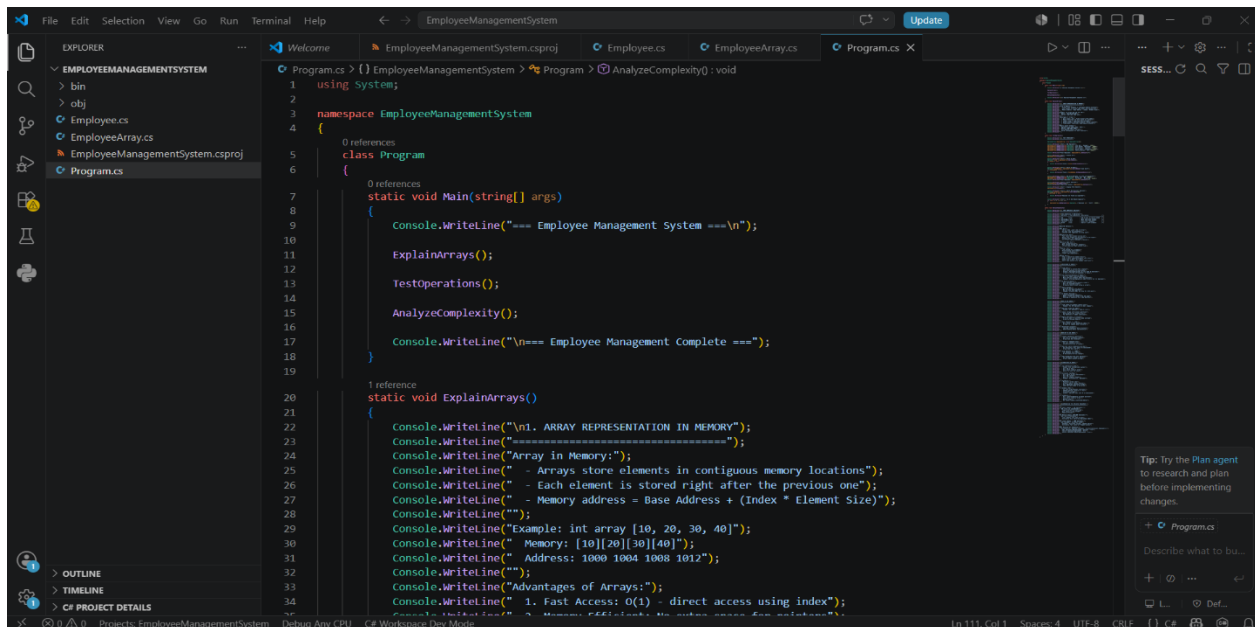


Hands-On

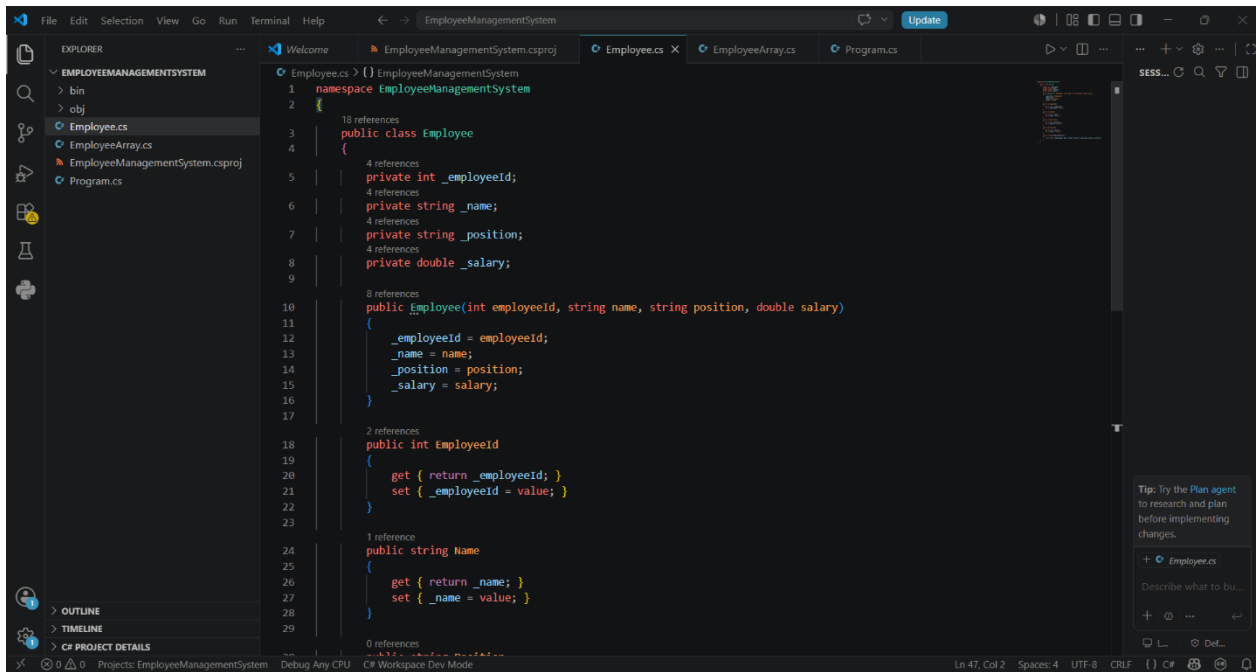
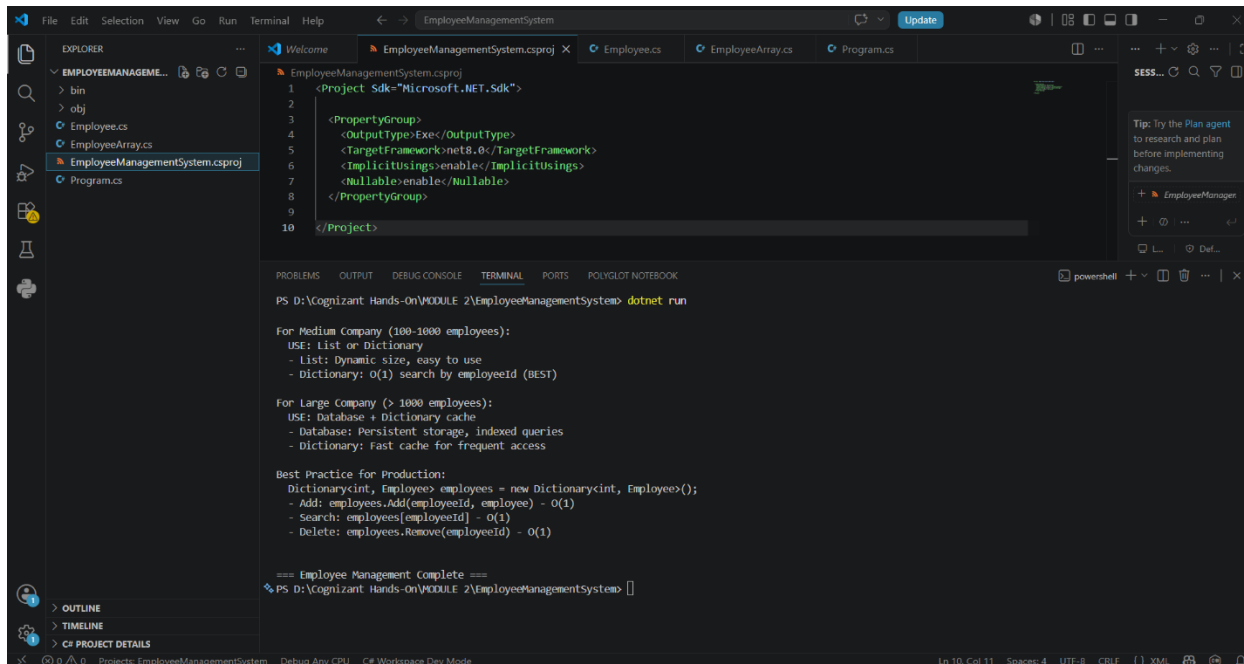
Exercise 4: Employee Management System

During this module, I successfully implemented, executed, and tested an Employee Management System in .NET by developing a project named EmployeeManagementSystem. I created an Employee class with attributes such as EmployeeId, Name, Position, and Salary to manage employee information. I used an array data structure to store employee records and implemented methods to add, search, traverse, and delete employee details. After implementation, I executed and tested various operations to ensure proper storage and management of records. I also analyzed the time complexity of different operations and understood the advantages and limitations of arrays. This practical exercise helped me understand the importance of arrays and selecting suitable data structures for developing efficient and maintainable .NET applications.

Coding:



```
1 using System;
2
3 namespace EmployeeManagementSystem
4 {
5     0 references
6     class Program
7     {
8         0 references
9         static void Main(string[] args)
10        {
11            Console.WriteLine("=== Employee Management System ===\n");
12
13            ExplainArrays();
14
15            TestOperations();
16
17            AnalyzeComplexity();
18
19            Console.WriteLine("\n=== Employee Management Complete ===");
20        }
21
22        1 reference
23        static void ExplainArrays()
24        {
25            Console.WriteLine("\n1. ARRAY REPRESENTATION IN MEMORY");
26            Console.WriteLine("=====");
27            Console.WriteLine("Array in Memory:");
28            Console.WriteLine(" - Arrays store elements in contiguous memory locations");
29            Console.WriteLine(" - Each element is stored right after the previous one");
30            Console.WriteLine(" - Memory address = Base Address + (Index * Element Size)");
31            Console.WriteLine("");
32            Console.WriteLine("Example: int array [10, 20, 30, 40]");
33            Console.WriteLine("Memory: [10][20][30][40]");
34            Console.WriteLine("Address: 1000 1004 1008 1012");
35            Console.WriteLine("");
36            Console.WriteLine("Advantages of Arrays:");
37            Console.WriteLine(" 1. Fast Access: O(1) - direct access using index");
38        }
39    }
40 }
```



Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant_Hands-On\MODULE 2\EmployeeManagementSystem> dotnet build
Restore complete (0.3s)
EmployeeManagementSystem net8.0 succeeded (2.0s) -> bin\Debug\net8.0\EmployeeManagementSystem.dll
Bui Focus folder in explorer (ctrl + click)
PS D:\Cognizant_Hands-On\MODULE 2\EmployeeManagementSystem> dotnet run
=== Employee Management System ===

1. ARRAY REPRESENTATION IN MEMORY
=====
Array in Memory:
- Arrays store elements in contiguous memory locations
- Each element is stored right after the previous one
- Memory address = Base Address + (Index * Element Size)

Example: int array [10, 20, 30, 40]
Memory: [10][20][30][40]
Address: 1000 1004 1008 1012

Advantages of Arrays:
1. Fast Access: O(1) - direct access using index
2. Memory Efficient: No extra space for pointers
3. Simple: Easy to understand and implement
4. Cache Friendly: Contiguous memory = better caching
5. Predictable: Fixed size, known memory location

Memory Layout Example:
Employee Array [Emp1, Emp2, Emp3]:
Address: 5000 5048 5096
Each Employee object: 48 bytes
Stored contiguously for fast traversal

2. TEST OPERATIONS
=====
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant_Hands-On\MODULE 2\EmployeeManagementSystem> dotnet run

Test 1: Add Employees
Employee added: ID: 1, Name: John Doe, Position: Manager, Salary: $75000
Employee added: ID: 2, Name: Jane Smith, Position: Developer, Salary: $65000
Employee added: ID: 3, Name: Bob Johnson, Position: Analyst, Salary: $55000
Employee added: ID: 4, Name: Alice Brown, Position: Developer, Salary: $70000
Employee added: ID: 5, Name: Charlie Wilson, Position: HR, Salary: $50000
Total Employees: 5

Test 2: Display All
=== All Employees ===
Total: 5
-----
ID: 1, Name: John Doe, Position: Manager, Salary: $75000
ID: 2, Name: Jane Smith, Position: Developer, Salary: $65000
ID: 3, Name: Bob Johnson, Position: Analyst, Salary: $55000
ID: 4, Name: Alice Brown, Position: Developer, Salary: $70000
ID: 5, Name: Charlie Wilson, Position: HR, Salary: $50000
-----

Test 3: Search by ID
Found: ID: 3, Name: Bob Johnson, Position: Analyst, Salary: $55000

Test 4: Search by Name
Found: ID: 2, Name: Jane Smith, Position: Developer, Salary: $65000

Test 5: Add More Employees (to show array growing)
Employee added: ID: 6, Name: Diana Lee, Position: QA, Salary: $48000
Employee added: ID: 7, Name: Eve Davis, Position: Marketing, Salary: $52000
Total Employees: 7

Test 6: Delete Employee
Employee deleted: ID: 3, Name: Bob Johnson, Position: Analyst, Salary: $55000
Total After Delete: 6
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\EmployeeManagementSystem> dotnet run

Test 8: Search Non-Existent Employee
Employee with ID 999 not found
Employee not found (as expected)

Test 9: Try to Add Beyond Capacity
Employee added: ID: 8, Name: Employee 8, Position: Staff, Salary: $40000
Employee added: ID: 9, Name: Employee 9, Position: Staff, Salary: $40000
Employee added: ID: 10, Name: Employee 10, Position: Staff, Salary: $40000
Employee added: ID: 11, Name: Employee 11, Position: Staff, Salary: $40000
Cannot add: Array is full (max 10 employees)
Cannot add: Array is full (max 10 employees)
Cannot add: Array is full (max 10 employees)
Cannot add: Array is full (max 10 employees)

3. TIME COMPLEXITY ANALYSIS
=====

Time Complexity of Operations:
| Operation | Time Complexity | Description |
|-----|-----|-----|
| Add | O(1) | Add at end, no search needed |
| Search(ID) | O(n) | Must check each element |
| Search(Name) | O(n) | Must check each element |
| Traverse | O(n) | Visit all n elements |
| Delete | O(n) | Search + shift elements |

Detailed Analysis:

Add: O(1)
- Add at index _count (end of array)
- No loop, just one assignment
- Constant time regardless of array size

Search by ID: O(n)
- Must check each element one by one
```